Unit 2: Python Multithreaded Programming

Sub-unit 2.1 — What is Multithreading?

Definition

Definition

Multithreading is a programming technique in which a single process is divided into two or more threads of execution, each running independently but sharing the same memory space, resources, and code of the parent process. It allows a program to perform multiple operations concurrently, improving responsiveness and, in suitable workloads, overall throughput.

Notes

A thread is often described as a "lightweight process" — it is the smallest unit of execution that an operating system scheduler can manage independently.

- Every process starts with one thread, called the main thread. A program can spawn additional threads from within this main thread.
- All threads that belong to the same process share the process's global data, heap memory, open file descriptors, and code segment.
- Each thread still keeps its own program counter, its own register set, and its own stack — this is what lets each thread execute independently even though memory is shared.
- Because threads share memory, communication between threads is easy (they can read/write the same variables directly) — but this same sharing is also the source of race conditions, which is why synchronization tools (locks, semaphores) exist.
- Multithreading is most useful for I/O-bound tasks (file access, network calls, user input) where a thread would otherwise sit idle waiting; it is less effective for CPU-bound tasks in CPython specifically (see GIL note below).
- Common real-world uses: responsive GUIs (keeping the interface active while a task runs in the background), web servers handling many client requests, downloading multiple files simultaneously, and background logging or monitoring.

Special Concept: The Global Interpreter Lock (GIL)

CPython (the standard Python interpreter) has a Global Interpreter Lock — a mutex that allows only one thread to execute Python bytecode at any given instant, even on a multi-core machine. This means Python threads give you true concurrency (tasks appear to progress together, and CPU sits idle less often during waits) but not true parallelism for CPU-bound Python code. For CPU-heavy work, the multiprocessing module (separate processes, separate memory, no shared GIL) is typically preferred; for I/O-bound work, threading remains an excellent and lightweight choice.

Diagram: A Process and Its Threads

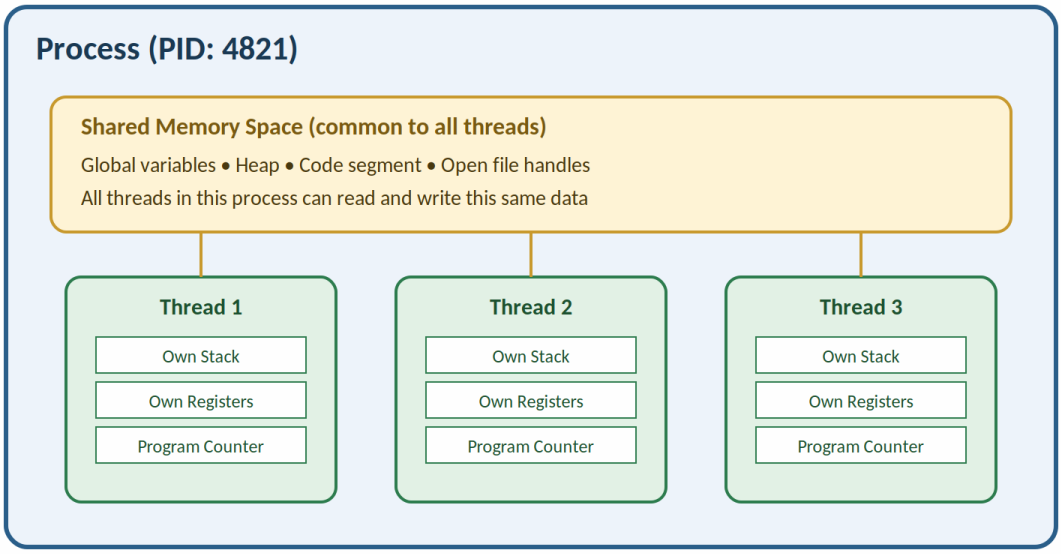


Figure 2.1: One process containing three threads. All threads share the process's memory, but each keeps its own stack, registers, and program counter.

Python Code Demonstration

The example below creates two threads from the main thread. One prints numbers, the other prints letters. Because both threads run concurrently, their output interleaves rather than appearing strictly one after another.

```
import threading
import time

def print_numbers():
    for i in range(1, 6):
        print(f"Number: {i}")
        time.sleep(0.5)

def print_letters():
    for letter in 'ABCDE':
        print(f"Letter: {letter}")
        time.sleep(0.5)

# Create two thread objects, each targeting a function
t1 = threading.Thread(target=print_numbers)
t2 = threading.Thread(target=print_letters)

# Start both threads -- they now run concurrently
t1.start()
t2.start()

# Wait for both threads to finish before continuing
t1.join()
t2.join()
```

```
print("Both threads have finished execution.")
```

Expected behaviour:

- t1.start() and t2.start() return almost immediately — they hand off execution to the new threads rather than blocking.
- Because both threads sleep for 0.5 seconds between prints, you will see "Number" and "Letter" lines interleaved in the output, not grouped together.
- t1.join() and t2.join() make the main thread wait until each of the two worker threads has completed, so the final print statement runs only after both are done.

Quick Recap

- Thread = lightweight unit of execution within a process; shares memory with sibling threads.
- Multithreading = running multiple threads concurrently within one process.
- Best suited to I/O-bound tasks; CPython's GIL limits CPU-bound parallel speed-up via threads.
- Python's threading module (threading.Thread, .start(), .join()) is the standard way to create and manage threads.